Save Point

Architecture

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

Quiz App AI is a local-only, mobile-first Flutter app. It keeps a notebook of folders and notes, uses a user-supplied AI API key to generate quiz questions from note content, and reviews those questions on a spaced-repetition schedule.

Relationship to the Python desktop app

This project began as a mobile rewrite of `Quiz-app-Ai`, an existing Tkinter/Python desktop app (a separate repo, not a dependency of this one). Nothing here imports or shares code with it (Dart and Python can't share a codebase), but a lot of the *design* carries over deliberately:

- The layering below mirrors that project's `core/` → `storage/` → `gui/controllers/` → `gui/screens/` split.
- `core/scheduling.dart`, `core/grading.dart`, and the `ai/` provider abstraction are direct conceptual ports of that project's Python equivalents, adjusted where the two apps' requirements actually differ (see `scheduling_algorithm.md` for the one deliberate behavior change).
- Where this project diverges from that one, it's called out explicitly rather than left as a silent surprise. See `future_work.md`.

Layering

```
lib/
├── core/           depends on: nothing (pure Dart, no I/O)
├── storage/        depends on: core
├── ai/             depends on: core
├── settings/       depends on: core
├── services/        depends on: core, storage, ai, settings
├── auth/           depends on: nothing
├── theme/          depends on: nothing but flutter/material
└── screens/        depends on: core, storage, ai, settings, services, auth, theme
```

`core/`: pure business logic, zero I/O.

- `models.dart`: plain Dart classes shared across every layer (`Folder`, `Note`, `Question`, `Choice`, `ReviewRecord`, `SrsCard`, `ReviewRating`, `Problem`, `ProblemAttempt`). Every field is `final`; nothing

outside `storage/` should mutate one and expect it to reach the database (see the file's own doc comment for why).
- `scheduling.dart`: FSRS scheduling, a pure wrapper around the `fsrs` package (see `scheduling_algorithm.md`).
- `reminders.dart`: when to send study reminders and what they say.
- `grading.dart`: local short-answer grading (normalized text/keyword overlap, no AI call).
- `text_extraction.dart`: `.txt` / `.md` / `.pdf` / `.docx` / `.pptx` to plain text.

`storage/`: a thin `sqflite` data-access layer, one repo class per table group (`FoldersRepo`, `NotesRepo`, `QuestionsRepo`, `ReviewsRepo`, `ProblemsRepo`). No ORM: `db.dart` holds the schema and a connection factory (`openAppDatabase()`, with `appDatabaseFactory()` picking the SQLite backend per platform: IndexedDB-backed on web, FFI on Windows/Linux, the `sqflite` plugin on Android/iOS/macOS); each repo takes a `Database` via constructor injection (so tests can pass a real in-memory SQLite database instead of a hand-written fake) and exposes plain create/get/list/update methods that take/return `core/models.dart` classes. Repos never contain business logic; that lives in `core/` or `services/`.

`ai/`: a provider-agnostic abstraction for AI-driven question generation. See `ai_providers.md`.

`settings/`: non-secret app settings (`app_settings.dart`, a JSON string in `shared_preferences`). The AI API key itself is *not* stored here; it goes through `flutter_secure_storage` directly from the Account screen, since it's a secret and `shared_preferences` isn't encrypted.

`services/`: orchestration that spans multiple repos/the AI layer:

- `question_generation_service.dart`: reads a note, calls the configured `AIProvider`, persists the returned questions.
- `data_export_service.dart`: dumps every table's raw rows to one JSON file for the Settings screen's "Export all data" action (a share sheet on mobile/desktop, a download on web). `exportFile()` returns the file name alongside the file, because `XFile.fromData` only keeps the name on web; elsewhere it's passed to the share sheet via `fileNameOverrides`.
- `reminder_service.dart`: plans study reminders from what's due and hands them to `flutter_local_notifications`, scheduled ahead on Android/iOS/macOS and shown while open on Linux/Windows.

`auth/`: `AuthService` and its one implementation, `QuillFinAuthService`: a QuillFin account (Clerk, OAuth with PKCE through the system browser, in `clerk_oauth.dart` and `browser_auth.dart`) or a device-only session.

`sync/`: offline-first sync with the Convex backend. `sync_store.dart` holds the local half (dirty rows out, pulled rows in), `sync_service.dart` the engine and its schedule, and `convex_client.dart` Convex's HTTP API. `cloud_session.dart` holds the signed-in session, `CloudScope` and the plan. See `accounts_and_sync.md`.

`config/`: `CloudConfig`, the build-time Clerk and Convex settings.

`screens/`: the actual widgets, organized by tab (`today/`, `notes/`, `tracker/`, `quiz/`, `account/`, `settings/`, `login/`, and `review/` for the shared rating buttons), plus `root_shell.dart` (the five-tab shell) and `app.dart` (the composition root: the one place that decides which `AuthService`, which repos, which screens are wired together).

End-to-end data flow

1. **Add a note**: `NotesRepo.create()` persists it under a `Folder`. No automatic keyword extraction or note-linking happens (unlike the Python desktop app), not ported yet, see `future_work.md`.
2. **Import a file**: `core/text_extraction.extractText()` turns a picked file's bytes into plain text; either a single import (straight into the note editor) or a bulk import (through `BulkImportStagingScreen`'s preview/reorder/confirm flow; nothing is written to the real notebook until the user confirms).
3. **Generate questions**: `QuestionGenerationService.generateForNote()` reads the note, calls the configured `AIProvider` (built via `ai/provider_factory.dart` from whatever provider/model/API key is configured), and persists each `GeneratedQuestion` via `QuestionsRepo.create()` as a new FSRS card, due straight away.
4. **Review**: the Today tab's "Start quiz" opens `QuizScreen`, which pulls everything due via `QuestionsRepo.listDue()` and grades the answer (`core/grading.dart` for short-answer, direct `Choice.isCorrect` equality for multiple-choice). The user confirms or changes the suggested rating; `core/scheduling.reviewCard()` computes the new card, persisted via `QuestionsRepo.updateScheduling()`, and an immutable row goes to `question_history` via `ReviewsRepo.record()`. 4b. **Practice a problem**: on the Tracker tab, `ProblemDetailScreen` logs an attempt (solved or not, time taken, self-rated) through `ProblemsRepo.recordAttempt()`, which moves the problem's own FSRS card. 4c. **Reminders**: after any review, attempt or settings change, `RootShell` re-plans reminders via `ReminderService.trySync()`.
5. **Edit a question**: `QuestionEditorScreen` lets you hand-edit an AI-generated question's prompt/answer/choices via `QuestionsRepo.updateContent()`, which marks `isManuallyEdited` so a future regeneration pass could tell "the user changed this on purpose" apart from "still exactly what the AI produced."
6. **Export**: `DataExportService.buildExport()` reads every table's raw rows directly (not through the typed models; see that file's doc comment for why) into one JSON structure, shared via the OS share sheet.

Composition root

`lib/app.dart::QuizApp` wires an `AuthService` and shows either the login screen or `RootShell`. `RootShell` (`lib/screens/root_shell.dart`) opens the SQLite connection and constructs the five tab screens (Today, Notes, Tracker, Account, Settings), driving navigation via an `IndexedStack` and a `NavigationBar`. It owns the `ReminderService` (passed in from `main.dart`) and a change counter: when one tab reports a study change, the others refresh and reminders are re-planned.